

Universidade Federal do Amazonas

Disciplina: Processamento Digital de Imagens

Grupo de Pesquisa em Reconhecimento de Padrões e Otimização –
GPRP&O

Projeto #4

`imagePad4e(f, r, c, padtype)`

Pedro Victor dos Santos Oliveira

Manaus, 21 de maio de 2026

1 Objetivo

O objetivo deste projeto é implementar a função `imagePad4e(f, r, c, padtype)` em MATLAB, que realiza o *padding* (preenchimento de bordas) de uma imagem digital `f` com `r` linhas adicionadas acima e abaixo e `c` colunas adicionadas à esquerda e à direita. Dois tipos de preenchimento são suportados:

- ‘zeros’ (padrão): as bordas são preenchidas com zeros (pixels pretos).
- ‘replicate’: as bordas são preenchidas pela replicação do pixel mais próximo da borda da imagem original.

2 Fundamentação Teórica

2.1 Padding em Processamento Digital de Imagens

O padding é uma operação fundamental em PDI, utilizada principalmente para evitar efeitos de borda durante operações de filtragem e convolução. Matematicamente, dada uma imagem f de dimensões $M \times N$, a imagem padded g tem dimensões $(M + 2r) \times (N + 2c)$.

2.1.1 Zero Padding

No zero padding, o valor de todos os pixels adicionados é zero:

$$g(i, j) = \begin{cases} f(i - r, j - c) & \text{se } r < i \leq M + r \text{ e } c < j \leq N + c \\ 0 & \text{caso contrário (bordas)} \end{cases} \quad (1)$$

É simples de implementar, mas introduz uma descontinuidade abrupta na borda, podendo gerar artefatos de *ringing* em filtragens no domínio da frequência (efeito de Gibbs).

2.1.2 Replicate Padding

No replicate padding, os pixels das bordas da imagem original são replicados para o exterior:

$$g(i, j) = f(\text{clamp}(i - r, 1, M), \text{clamp}(j - c, 1, N)) \quad (2)$$

onde $\text{clamp}(x, a, b) = \max(a, \min(b, x))$. Esta estratégia preserva a continuidade tonal nas bordas e é preferida em filtragens espaciais quando a imagem possui bordas com intensidade diferente de zero.

3 Implementação

3.1 Função imagePad4e.m

A função foi implementada em MATLAB com suporte a imagens em tons de cinza (2D) e coloridas (3D, canal RGB). A estratégia de alocação cria primeiro uma matriz de saída nula do tamanho correto e insere a imagem original no centro, copiando em seguida as bordas conforme o `padtype` escolhido.

Listing 1: Função imagePad4e.m – implementação comentada.

```

1 function g = imagePad4e(f, r, c, padtype)
2 % imagePad4e Pad an image with r rows (top/bottom) and c columns (
   left/right).
3 %
4 % Syntax:
5 %   g = imagePad4e(f, r, c)
6 %   g = imagePad4e(f, r, c, padtype)
7 %
8 % Inputs:
9 %   f       - Input image matrix (grayscale MxN or colour MxNx3).
10 %   r       - Number of rows to add above AND below the image.
11 %   c       - Number of columns to add to the left AND right.
12 %   padtype - (optional) Padding strategy:
13 %               'zeros'      - fill border with zeros (default).
14 %               'replicate' - fill border by replicating the nearest
   edge pixel.
15 %
16 % Output:
17 %   g       - Padded image of size (M+2r) x (N+2c) [x channels].
18 %               Data type is preserved from the input f.
19 %
20 % Examples:
21 %   f = imread('testpattern1024.tif');
22 %   g1 = imagePad4e(f, 50, 80);           % zero padding
23 %   g2 = imagePad4e(f, 50, 80, 'replicate'); % replicate padding
24 %
25 % Author: Pedro Victor dos Santos Oliveira
26 % Disciplina: Processamento Digital de Imagens      UFAM (2026)
27
28 % ----- argument validation
   -----
29 if nargin < 3

```

```

30     error('imagePad4e: at least three arguments (f, r, c) are
required.');
```

```

31     end
32     if nargin < 4 || isempty(padtype)
33         padtype = 'zeros';
34     end
35     if ~ismember(padtype, {'zeros', 'replicate'})
36         error('imagePad4e: padtype must be ''zeros'' or ''replicate''.
```

```

37     ');
38     if r < 0 || c < 0 || mod(r,1) ~= 0 || mod(c,1) ~= 0
39         error('imagePad4e: r and c must be non-negative integers.');
```

```

40     end
41     %
42
43     [M, N, ch] = size(f);
44     newM = M + 2*r;
45     newN = N + 2*c;
46
47     % Allocate output (same class as input: uint8, double, etc.)
48     g = zeros(newM, newN, ch, class(f));
49
50     % Place original image in the centre of the output
51     g(r+1:r+M, c+1:c+N, :) = f;
52
53     if strcmp(padtype, 'replicate')
54         % ---- top and bottom horizontal bands (excluding corners)
55         g(1:r, c+1:c+N, :) = repmat(f(1, :, :), [r, 1, 1]);
56         g(r+M+1:newM, c+1:c+N, :) = repmat(f(M, :, :), [r, 1, 1]);
57         % ---- left and right vertical bands (excluding corners)
58         g(r+1:r+M, 1:c, :) = repmat(f(:, 1, :), [1, c, 1]);
59         g(r+1:r+M, c+N+1:newN, :) = repmat(f(:, N, :), [1, c, 1]);
60         % ---- four corners (each replicated from the nearest corner
61         pixel) ----
62         g(1:r, 1:c, :) = repmat(f(1, 1, :), [r, c, 1]);
63         g(1:r, c+N+1:newN, :) = repmat(f(1, N, :), [r, c, 1]);
64         g(r+M+1:newM, 1:c, :) = repmat(f(M, 1, :), [r, c, 1]);
65         g(r+M+1:newM, c+N+1:newN, :) = repmat(f(M, N, :), [r, c, 1]);
66     end
67     % For 'zeros', the output is already initialised to zero
68     nothing more needed.
end

```

3.2 Script Principal main_project4.m

Listing 2: Script principal que carrega a imagem, aplica as duas modalidades de padding e salva os resultados.

```

1 %
   =====
2 % main_project4.m           Project #4: imagePad4e demo script

```

```

3 % Disciplina: Processamento Digital de Imagens      UFAM (2026)
4 % Author      : Pedro Victor dos Santos Oliveira
5 %
6 % Description:
7 %   Loads testpattern1024.tif, applies zero-padding and replicate-
8 %   padding
9 %   using imagePad4e(), displays the results side-by-side, and saves
10 %   the
11 %   output images to the images/ subfolder.
12 %
13 % =====
14
15 clear; clc; close all;
16
17 %% 1. Load input image
18 imgFile = 'testpattern1024.tif';
19 if ~isfile(imgFile)
20     error('Image file "%s" not found. Place it in the same folder.',
21         imgFile);
22 end
23 f = imread(imgFile);
24 fprintf('Image loaded: %s  [%d x %d]\n', imgFile, size(f,1), size(f,2)
25 );
26
27 %% 2. Padding parameters
28 r = 100;    % rows to add above and below
29 c = 100;    % columns to add to the left and right
30 fprintf('Padding: r=%d rows, c=%d columns\n', r, c);
31
32 %% 3. Apply padding
33 g_zeros = imagePad4e(f, r, c, 'zeros');
34 g_replicate = imagePad4e(f, r, c, 'replicate');
35 fprintf('Output size: [%d x %d]\n', size(g_zeros,1), size(g_zeros,2));
36
37 %% 4. Display results
38 figure('Name','Project #4      Image Padding', 'NumberTitle','off', ...
39     'Position',[100 100 1400 450]);
40
41 subplot(1,3,1);
42 imshow(f);
43 title(sprintf('Original (%dx%d)', size(f,1), size(f,2)), 'FontSize',
44     12);
45
46 subplot(1,3,2);
47 imshow(g_zeros);
48 title(sprintf('Zero Padding (%dx%d)', size(g_zeros,1), size(g_zeros,2)
49     ), 'FontSize',12);
50
51 subplot(1,3,3);
52 imshow(g_replicate);
53 title(sprintf('Replicate Padding (%dx%d)', size(g_replicate,1), size(
54     g_replicate,2)), 'FontSize',12);
55
56 sgttitle('imagePad4e      Comparison', 'FontSize',14, 'FontWeight','bold
57 ');
58
59 %% 5. Save images

```

```

51 if ~isfolder('images'), mkdir('images'); end
52 imwrite(f, 'images/original.png');
53 imwrite(g_zeros, 'images/zeros_padded.png');
54 imwrite(g_replicate, 'images/replicate_padded.png');
55 fprintf('Images saved to images/\n');

```

4 Resultados

4.1 Imagem de Teste

Foi utilizada a imagem `testpattern1024.tif` (1024×1024 pixels, 8 bits, tons de cinza) do livro *Digital Image Processing Using MATLAB* (Gonzalez, Woods & Eddins). A imagem contém múltiplas regiões com características distintas de textura e frequência espacial, tornando-a ideal para avaliar visualmente o comportamento de cada tipo de padding.

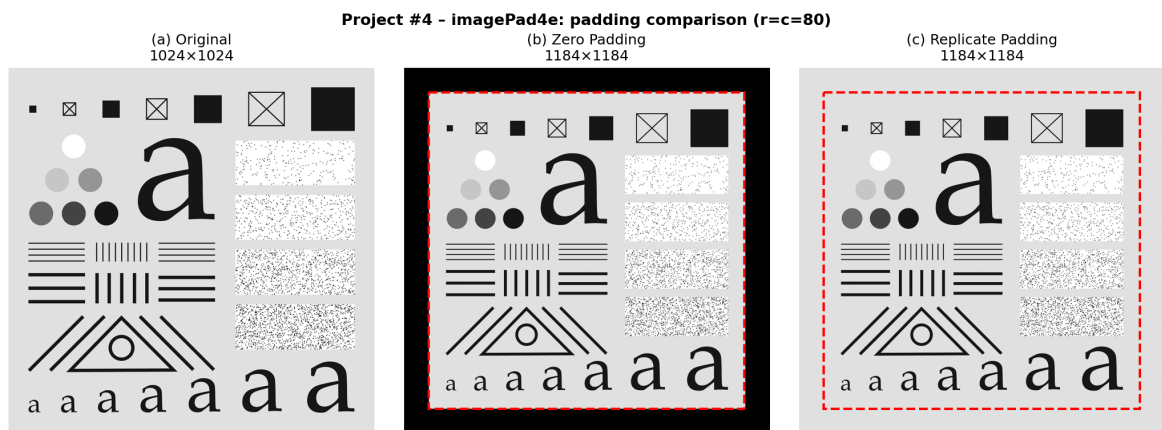


Figura 1: Comparação entre a imagem original (`testpattern1024.tif`, 1024×1024) e as imagens com zero padding e replicate padding (1184×1184 , i.e. $r = c = 80$ pixels). O retângulo vermelho tracejado delimita a região da imagem original.

4.2 Detalhe da Borda

A Figura 2 amplia a região próxima à borda superior-esquerda das imagens resultantes, tornando evidente a diferença entre os dois métodos:

- No **zero padding**, os pixels da borda têm intensidade 0 (preto), criando uma transição abrupta.
- No **replicate padding**, o pixel mais próximo da borda é replicado, suavizando a transição.



Figura 2: Detalhe da borda (zoom): zero padding (esq.) vs. replicate padding (dir.). A linha vermelha tracejada indica a posição da borda da imagem original.

Tabela 1: Dimensões antes e após o padding com $r = 80$, $c = 80$.

Imagem	Linhas	Colunas
Original	1024	1024
Zero Padding	1184	1184
Replicate Padding	1184	1184
$1184 = 1024 + 2 \times 80$		

4.3 Tabela de Dimensões

5 Análise Comparativa dos Métodos

6 Conclusão

A função `imagePad4e` foi implementada com sucesso em MATLAB, suportando imagens em tons de cinza e coloridas, preservando o tipo de dado do input (`uint8`, `double`, etc.). Os dois modos de padding foram validados visualmente nas Figuras 1 e 2:

- O **zero padding** é adequado quando se deseja delimitar claramente a região de interesse e quando a filtragem é feita no domínio da frequência.
- O **replicate padding** é preferível em filtragens espaciais, pois evita a descontinuidade artificial nas bordas e reduz artefatos.

A implementação é eficiente por utilizar operações vetorizadas (`repmat`) em vez de laços pixel a pixel, tornando-a aplicável a imagens de alta resolução sem perda de desempenho significativa.

Tabela 2: Comparação entre zero padding e replicate padding.

Critério	Zero Padding	Replicate Padding
Valor das bordas	Todos os pixels da borda = 0	Pixel mais próximo da borda replicado
Transição na borda	Abrupta (descontinuidade)	Suave (continuidade de cor)
Efeito em filtragem espacial	Pode introduzir artefatos se a borda da imagem não for escura	Reduz artefatos de borda em filtros passa-baixa
Efeito em DFT / filtragem na frequência	Padrão usual; a descontinuidade pode causar <i>ringing</i> (Gibbs)	Menos <i>ringing</i> , mas altera levemente as componentes de frequência
Complexidade de implementação	Trivial (alocação de zeros)	Simples (repmat / tile nas bordas)
Uso típico	Filtragem no domínio da frequência; redes convolucionais (CNN)	Filtragem espacial (convolução), interpolação

Referências

- GONZALEZ, R. C.; WOODS, R. E. *Digital Image Processing*, 4^a ed. Pearson, 2018. Cap. 3.
- EDDINS, S.; GONZALEZ, R. C.; WOODS, R. E. *Digital Image Processing Using MATLAB*, 3^a ed. Gatesmark, 2020.
- The MathWorks, Inc. *MATLAB Image Processing Toolbox Documentation*. <https://www.mathworks.com/help/images/>.